



ANÁLISE DO IMPACTO DO TAMANHO MÉDIO DOS COMMITTS NA MANUTENIBILIDADE DE SISTEMAS DE SOFTWARE EM PYTHON

Eduardo Henrique Moreira dos Santos, Ian dos Reis Novais, João
Antonio Nascimento Pires, João Guilherme Falante Araújo, Mauricio
Fernandes Leite, Nalanda Luiza dos Santos, Roberta Sophia Carvalho
Silva



CONTEXTO



- ✓ A manutenção é uma das atividades mais importantes no ciclo de vida do software, especialmente em projetos colaborativos.
 - ✓ Em plataformas como o GitHub, as mudanças no código são registradas por meio de commits.
 - ✓ O tamanho dessas mudanças pode influenciar fatores como revisão de código, ocorrência de bugs e eficiência na manutenção dos sistemas.
- 

CONTEXTO



COMMIT

Commit é a ação pela qual um desenvolvedor contribui código para o repositório. Ele consiste em modificações em arquivos, como adicionar, remover ou alterar código. (Arafat; Riehle, 2009)

PULL REQUEST

Pull request é uma solicitação feita pelo contribuinte para que as mudanças desenvolvidas em um fork sejam revisadas e, se aprovadas, mescladas no repositório principal do projeto. (Gousios; Storey; Bacchelli, 2016)

CONTEXTO



BIC

Bug-introducing commit: É um commit que, numa correção futura de bug, foi identificado como a mudança que introduziu o problema, usando o algoritmo SZZ, que rastreia o histórico do código. (Śliwerski; Zimmermann; Zeller, 2005)

LOC

Representa o número total de linhas de código-fonte de um software, sendo uma métrica usada para medir o tamanho, custo e esforço de desenvolvimento

CONTEXTO



COMMIT PEQUENO

- Poucos arquivos modificados (até 5)
 - Baixa quantidade de linhas alteradas (até 25)
- (Hattori; Lanza, 2008)

COMMIT MÉDIO

- Modificam muitos arquivos (> 5), mas com linhas controladas ($26 \leq 125$); **OU** alteram poucos arquivos (≤ 5), porém com volume maior de linhas (> 25).

COMMIT GRANDE

- Muitos arquivos modificados: Mais de 5
 - Alta quantidade de linhas alteradas: Acima de 125
- (Hattori; Lanza, 2008)

PROBLEMA



- ✓ Commits são unidades fundamentais de mudança em sistemas versionados.
 - ✓ Commits muito grandes podem dificultar revisão, reduzir rastreabilidade e elevar risco de defeitos.
 - ✓ Diante disso, surge a necessidade de investigar como o tamanho dos commits se relaciona com indicadores de manutenção do código.
- 



GOAL

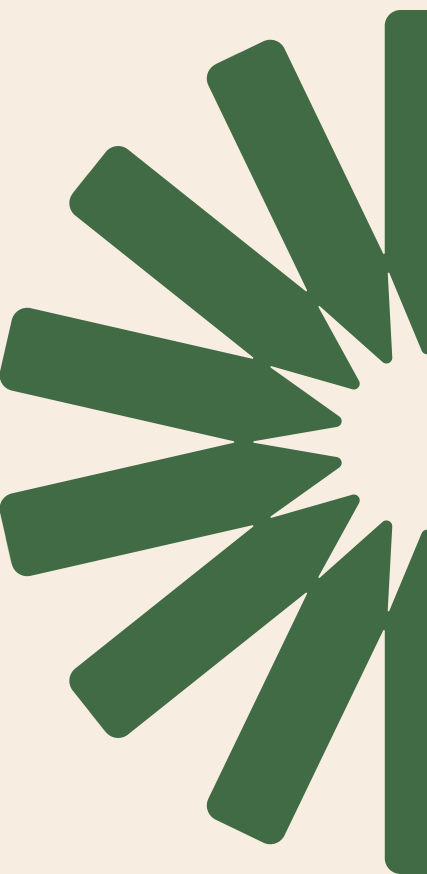
Analisar a associação entre o tamanho dos commits e seu impacto na manutenibilidade do software, considerando indicadores de manutenção, revisão de código e correção de defeitos, sob a perspectiva de mantenedores, no contexto de projetos open source populares desenvolvidos em Python no GitHub.



DELIMITAÇÃO



- Python como linguagem principal
- Janela temporal de 5 anos de repositórios
- Mais de 4 contribuintes
- Amostra de 500 repositórios
- Pelo menos 100 commits
- O repositório deve ter commits dentro desse último ano
- Apenas commits com 1 ano ou menos serão validados
- Priorização por popularidade (mais estrelas)



JUSTIFICATIVA DO RECORTE EXPERIMENTAL



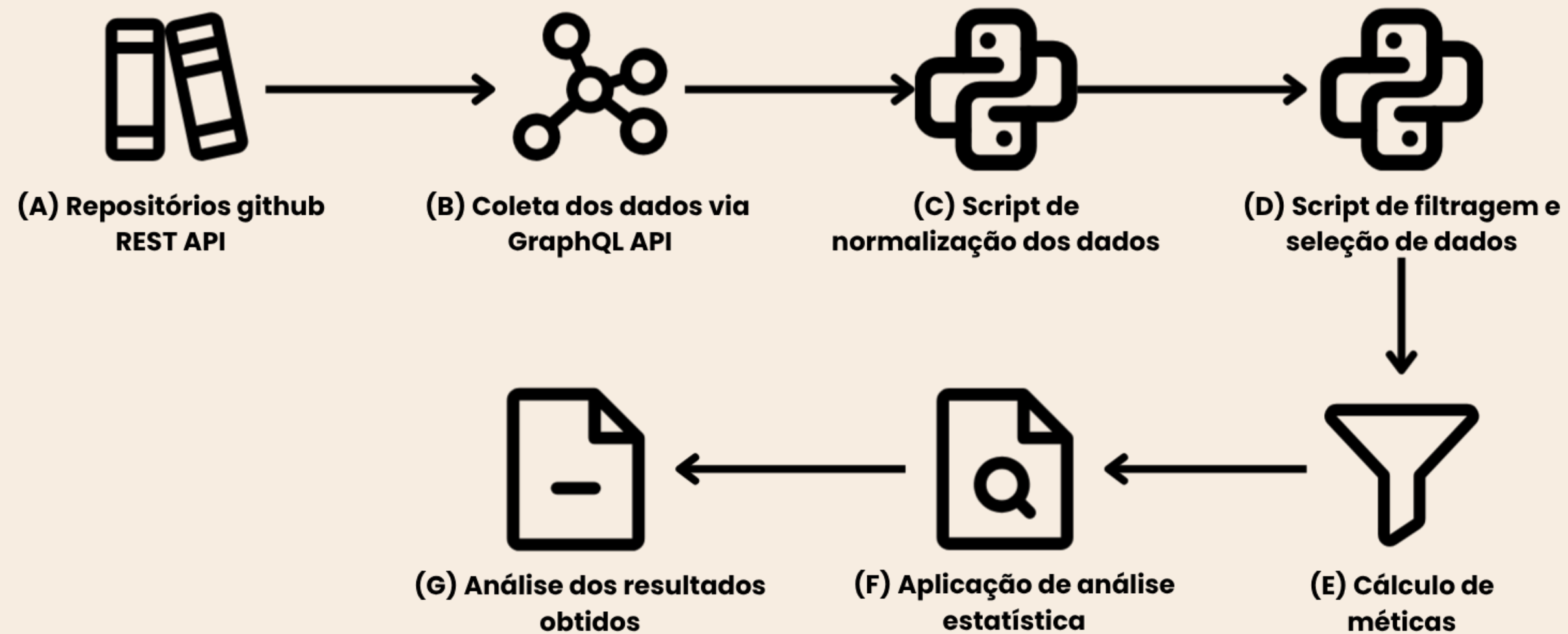
- MSR sobre GitHub com curadoria de projetos ativos e colaborativos (Kalliamvakou et al., 2015; Munaiah et al., 2017)
- Amostra (500 repositórios): equilíbrio entre validade e viabilidade (Ray et al., 2014; Cosentino et al., 2016)
- Janela de 5 anos + atividade recente: foco em manutenção real (Ait et al., 2022; Cosentino et al., 2016)

METODOLOGIA



- A. Seleção dos 500 repositórios Python mais ativos no GitHub, aplicando os critérios definidos no estudo.
- B. Extração de dados de commits, Pull Requests e issues dos repositórios selecionados utilizando a GitHub API.
- C. Organização, limpeza, padronização dos dados coletados utilizando scripts em Python.
- D. Filtragem e seleção dos dados relevantes com base em critérios definidos no estudo.
- E. Cálculo de métricas relacionadas à manutenção, como tempo de resolução de issues, revisões em PRs e reversões de commits.
- F. Aplicação de análise estatística para investigar correlações entre o tamanho dos commits e os indicadores de manutenção.
- G. Análise dos resultados obtidos para responder às questões de pesquisa definidas no modelo GQM.

METODOLOGIA



Q1: QUAL A RELAÇÃO ENTRE O TAMANHO DO COMMIT E A OCORRÊNCIA DE BUGS?

M1.1: LOC modificadas por commit (linhas adicionadas + removidas)

M1.2: Classificação do commit como bug-introducing (BIC), identificado pelo algoritmo MA-SZZ¹ a partir dos bug-fix commits via git blame

M1.3: Taxa de BICs por repositório ($n_BIC / total_commits$)

M1.4: Tamanho de efeito (Cliff's Delta) entre LOC dos BICs vs commits não bug-introducing

Hipótese

Commits que introduzem bugs tendem a ser maiores em linhas alteradas do que os demais.

Mudanças volumosas envolvem mais código, mais arquivos modificados de uma só vez e mais carga cognitiva para quem escreve e revisa, abrindo espaço para efeitos colaterais que passam despercebidos.

Q1: QUAL A RELAÇÃO ENTRE O TAMANHO DO COMMIT E A OCORRÊNCIA DE BUGS?

Analizamos 98.517 commits Python em 58* repositórios do GitHub para verificar se o tamanho do commit (em linhas de código alteradas) está associado à introdução de bugs.

5.223 BICS (5,3%)

93.294 COMMITS NORMAIS

Análise

- Mann-Whitney U (H_1 : BICs $>$ não-BICs em LOC)
- Cliff's Delta (Romano et al., 2006) = magnitude do efeito
- Spearman ρ por repositório = correlação LOC médio $\times$ taxa de BIC



Q1: QUAL A RELAÇÃO ENTRE O TAMANHO DO COMMIT E A OCORRÊNCIA DE BUGS?

pytorch/pytorch falhou por incompatibilidade de filesystem (NTFS case-insensitive);

AutoGPT apresentou erros repetidos de execução. Ambos foram removidos, resultando em 58 repositórios analisados.

Impacto marginal: cada repositório representa ~100 fixes sobre 10.000+ totais.



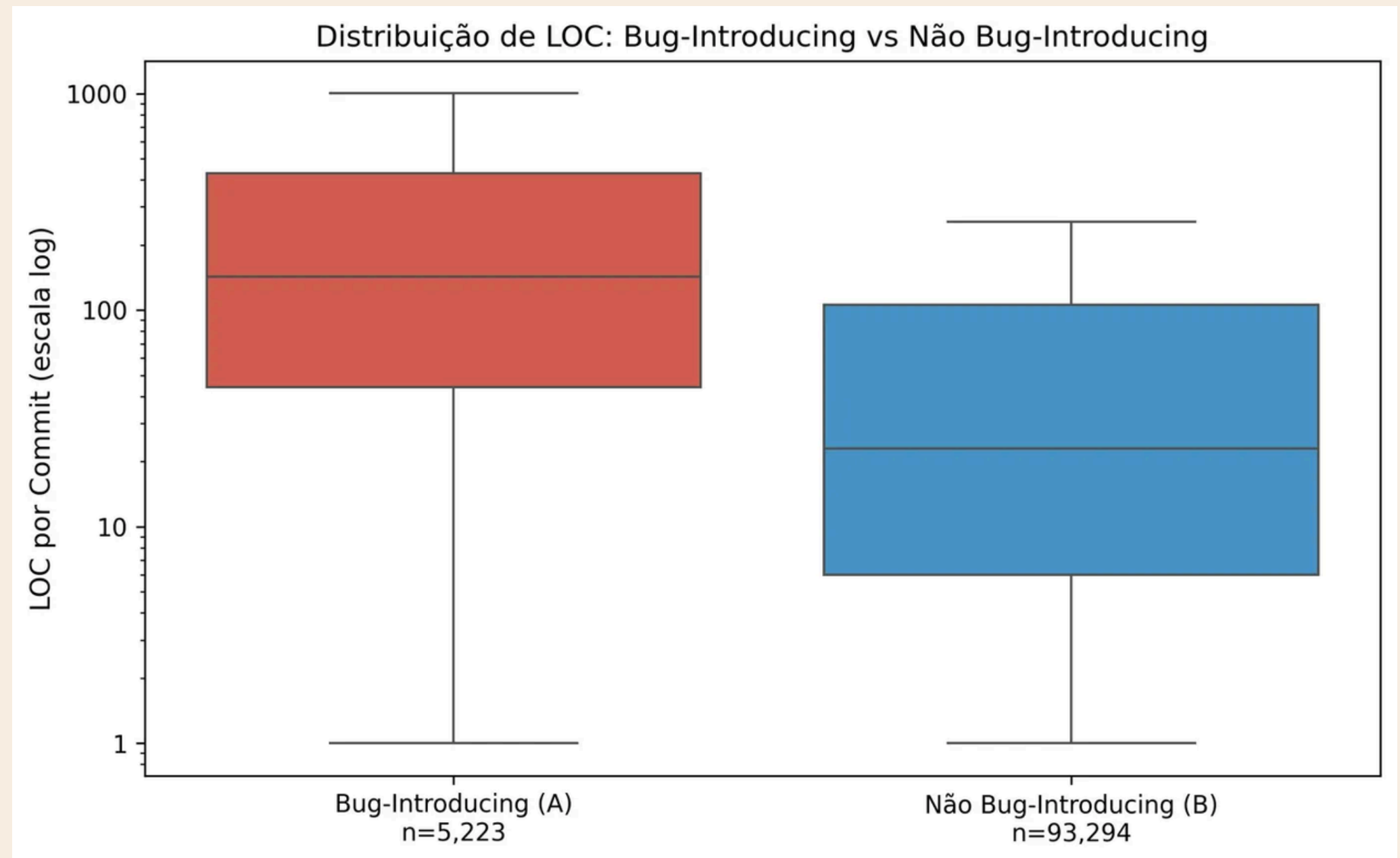
Q1: QUAL A RELAÇÃO ENTRE O TAMANHO DO COMMIT E A OCORRÊNCIA DE BUGS?

BICs são muito maiores que commits normais

143 LOC
MEDIANA DOS BICS

23 LOC
MEDIANA DOS
COMMITTS NORMAIS

6,2×
DIFERENÇA ENTRE
AS MEDIANAS



Q1: QUAL A RELAÇÃO ENTRE O TAMANHO DO COMMIT E A OCORRÊNCIA DE BUGS?

O que é Mann-Whitney?

Um teste que verifica se dois grupos têm distribuições diferentes, sem assumir que os dados seguem curva normal. O p-valor ínfimo indica que a diferença não é obra do acaso.

O que é Cliff's Delta?

Mede o tamanho prático do efeito: se sortearmos um BIC e um commit normal aleatoriamente, em $\sim 74\%$ das vezes o BIC será maior. Valores acima de 0,474 são classificados como efeito "grande".

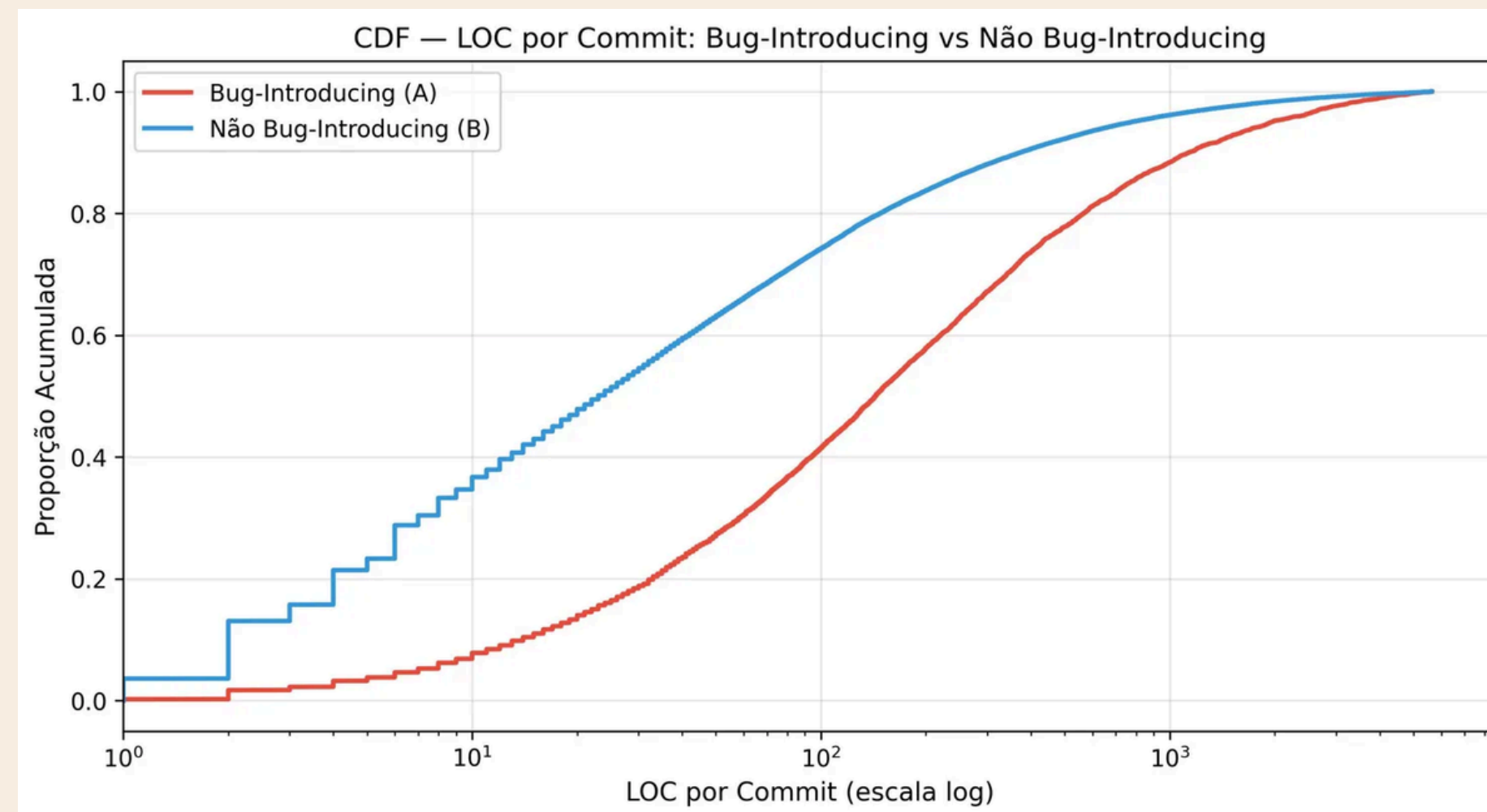
O que é Spearman ρ ?

Mede se duas variáveis crescem juntas (positivo) ou em sentidos opostos (negativo). $\rho = 0,36$ indica correlação moderada positiva: repositórios onde os commits são maiores em média tendem a ter uma fração maior de BICs.

Q1: QUAL A RELAÇÃO ENTRE O TAMANHO DO COMMIT E A OCORRÊNCIA DE BUGS?

MANN-WHITNEY $U \rightarrow P < 10^{-50}$

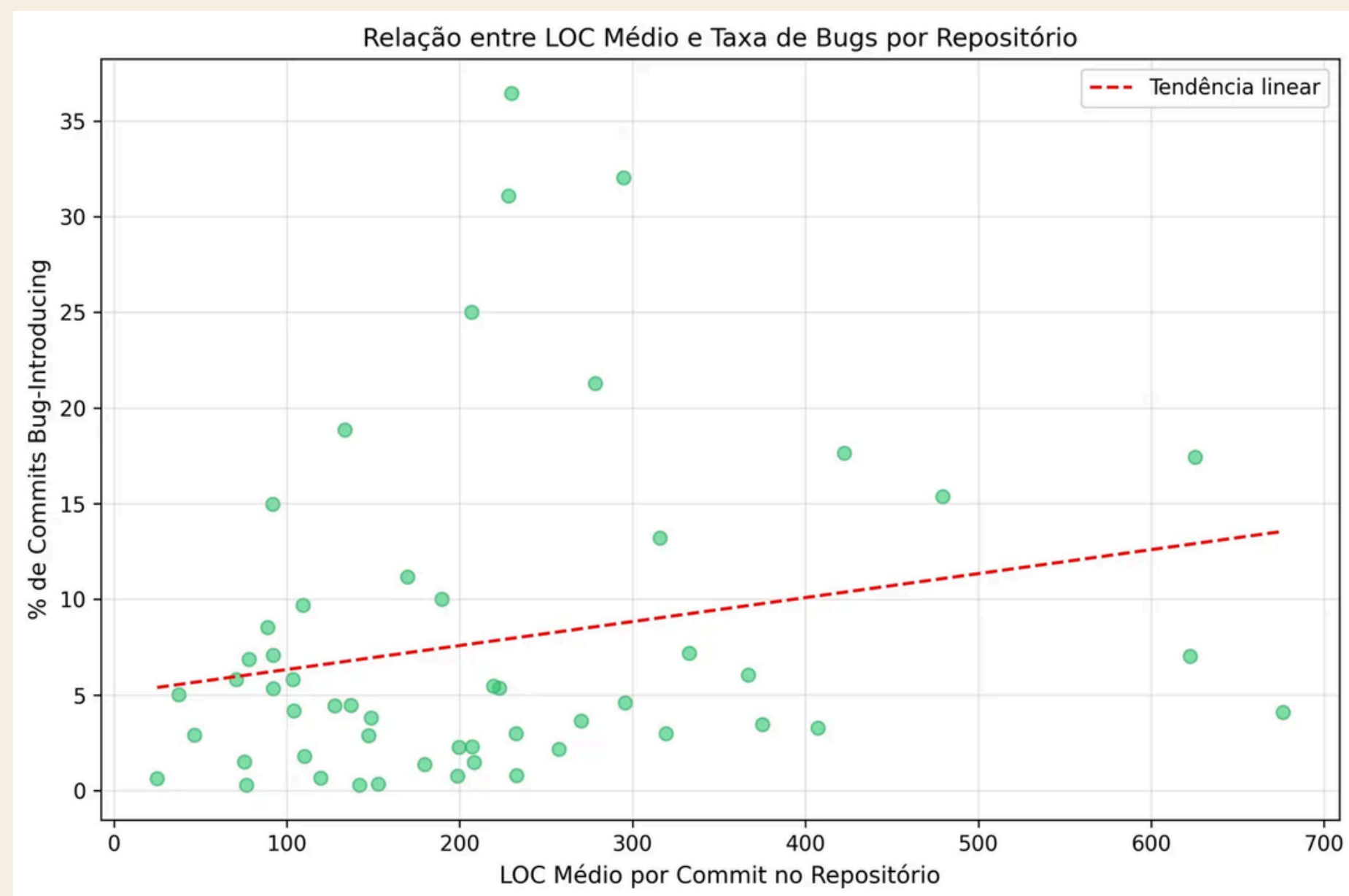
CLIFF'S DELTA = 0,48 $\rightarrow$ MAGNITUDE GRANDE



Q1: QUAL A RELAÇÃO ENTRE O TAMANHO DO COMMIT E A OCORRÊNCIA DE BUGS?

$P = 0,36$
CORRELAÇÃO DE SPEARMAN

$P = 0,006$
SIGNIFICÂNCIA



Q1: QUAL A RELAÇÃO ENTRE O TAMANHO DO COMMIT E A OCORRÊNCIA DE BUGS?

Hipótese Comprovada: BICs têm mediana de LOC 6,2× maior que commits normais (143 vs 23 LOC), com efeito de magnitude grande (Cliff's Delta = 0,48). O padrão se repete no nível dos projetos: repositórios com commits maiores têm proporcionalmente mais BICs ($\rho = 0,36$, $p = 0,006$).

Q2: O TAMANHO DO PR INFLUENCIA A COMPLEXIDADE DA REVISÃO DE CÓDIGO?

Q2 — Contexto da amostra

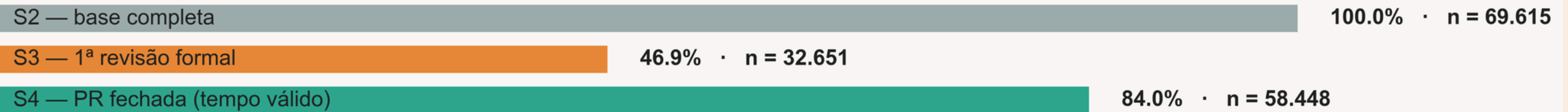
69.615

Pull requests (S2)

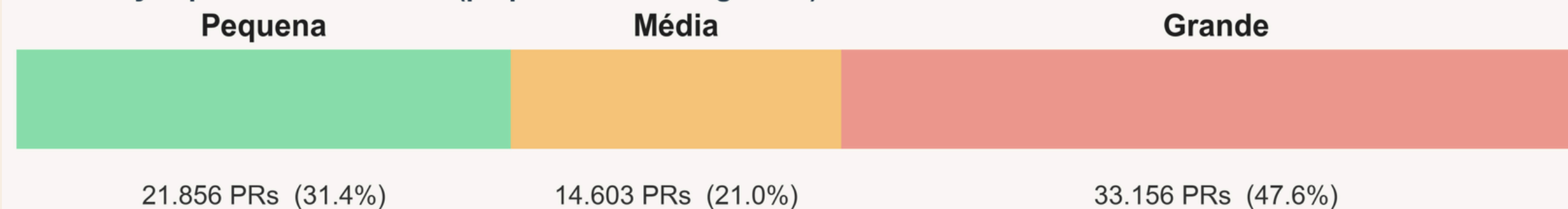
434

Repositórios Python populares

Subconjuntos — gráficos de tempo (M2.4 e M2.5)



Distribuição por tamanho da PR (pequena · média · grande)



Q2: O TAMANHO DO PR INFLUENCIA A COMPLEXIDADE DA REVISÃO DE CÓDIGO?

M2.1: LOC por PR (linhas adicionadas + removidas)

M2.2: Número de arquivos alterados por PR

M2.3: Comentários por PR (comments + review_comments)

$M2.3' = M2.3 \div LOC$ (densidade de discussão)

M2.4: Tempo até a primeira revisão da PR (first_review_at - created_at, em horas)

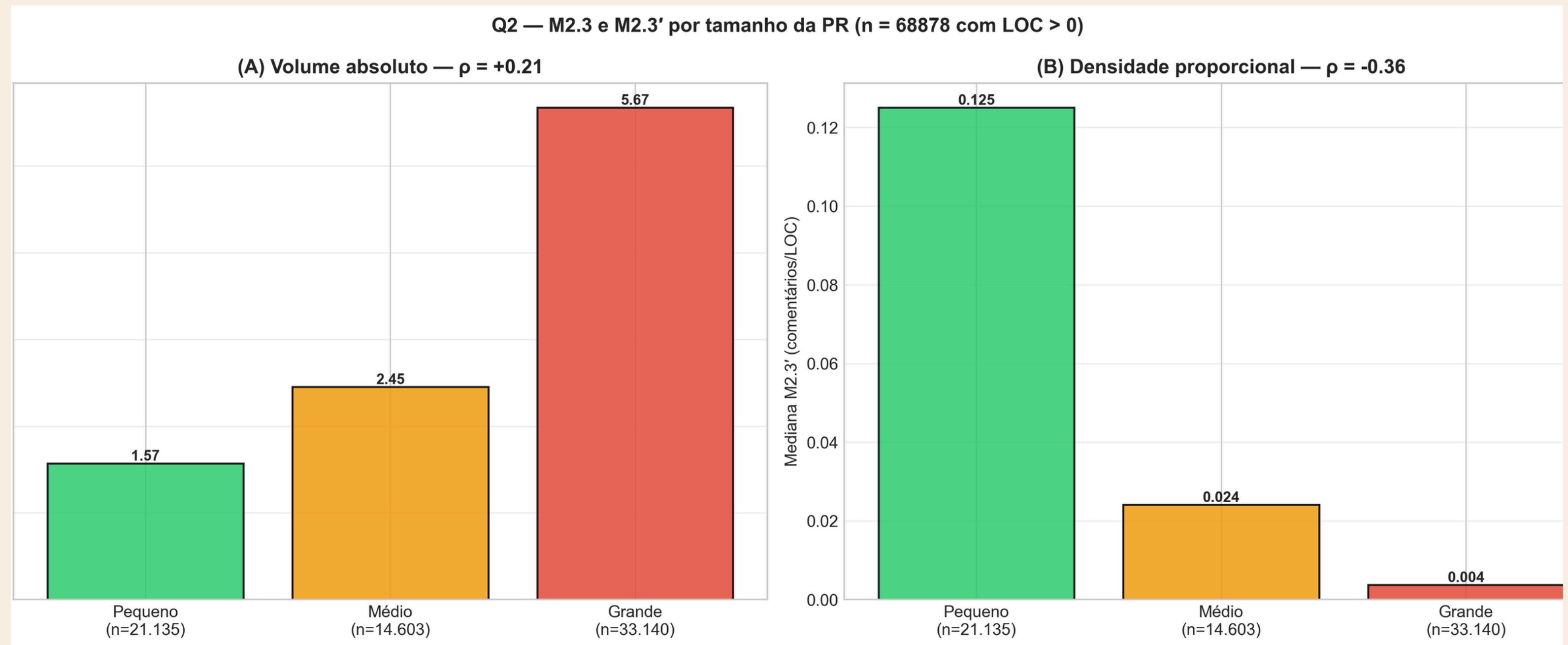
M2.5: Tempo até fechamento (closed_at - created_at, h)

Somente S4 (PR fechada, tempo válido)

Hipótese

PRs maiores tendem a gerar mais comentários e podem aumentar o tempo até a 1ª revisão e até o fechamento.

Q2: O TAMANHO DO PR INFLUENCIA A COMPLEXIDADE DA REVISÃO DE CÓDIGO?



Amostra com LOC > 0 (n = 68.878).

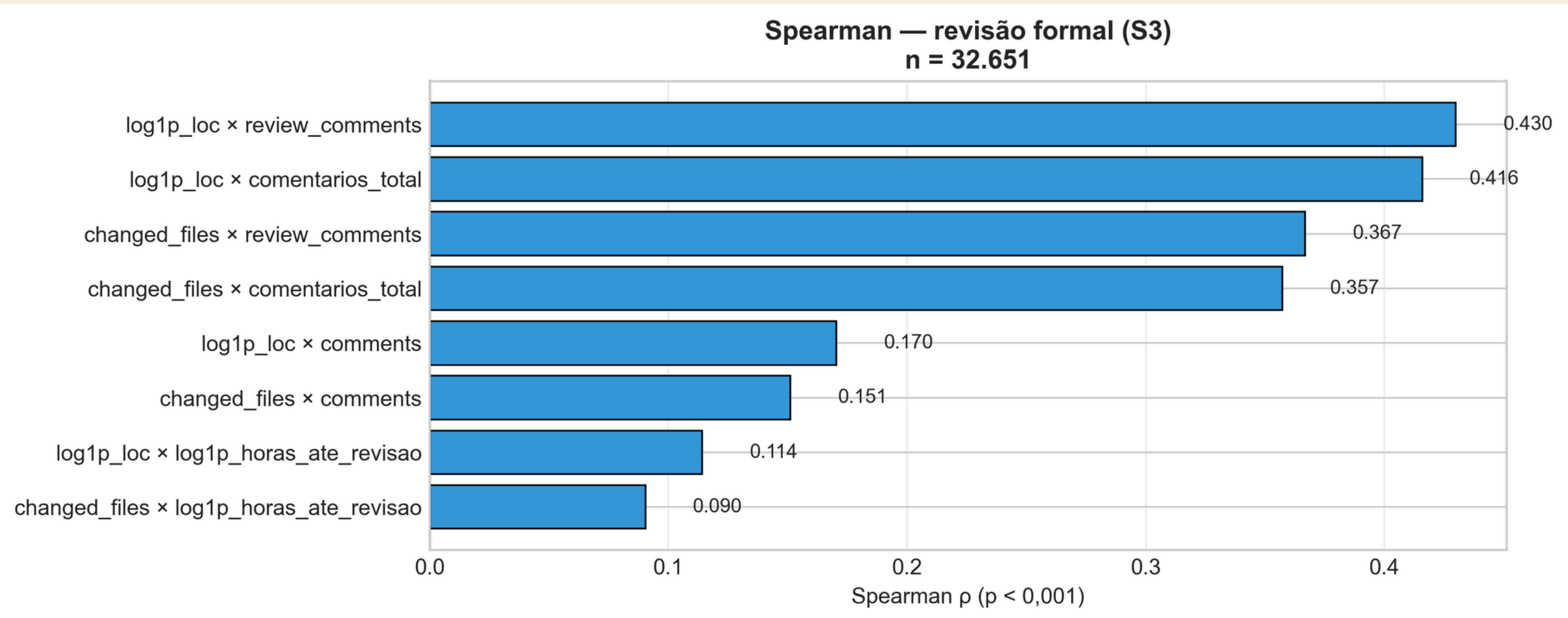
Pequeno: LOC ≤ 25 e ≤ 5 arquivos; grande: LOC ≥ 125 ou > 5 arquivos.

M2.3 = comments + review_comments; M2.3' = M2.3 $\div$ LOC

Mais comentários no total (A), porém menos comentários por linha (B) — volume de interação não equivale a revisão proporcionalmente mais profunda

Q2: O TAMANHO DO PR INFLUENCIA A COMPLEXIDADE DA REVISÃO DE CÓDIGO?

Associações em PRs com revisão formal (S3)

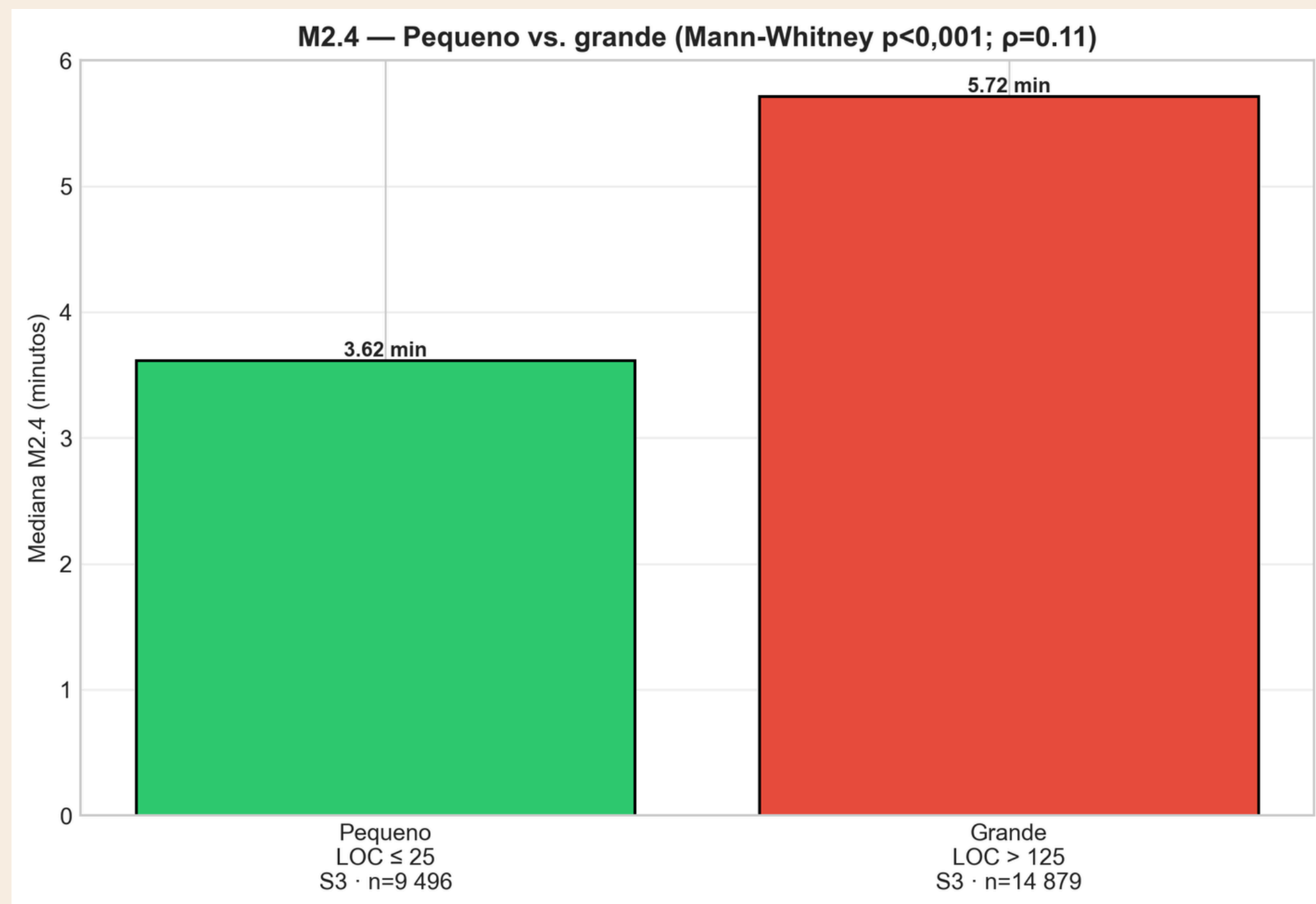


S3 = PRs com first_review_at na API (n = 32.651; ~46,9% da base). Todos os ρ com p < 0,001.

O tamanho explica sobretudo a discussão, não o atraso inicial da revisão formal; o escrutínio por linha continua menor em PRs grandes.

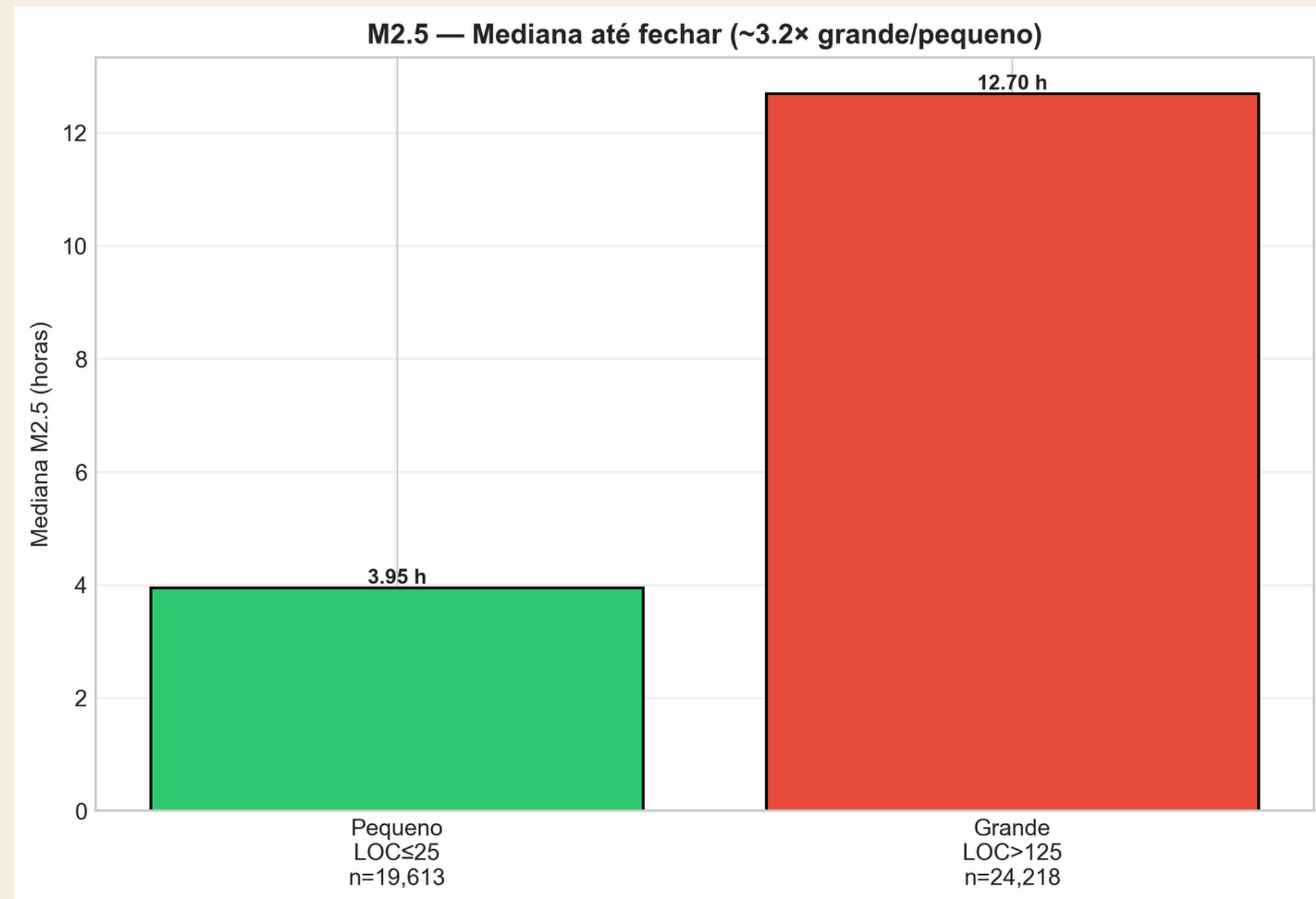
Q2: O TAMANHO DO PR INFLUENCIA A COMPLEXIDADE DA REVISÃO DE CÓDIGO?

Latência até a 1ª revisão formal (M2.4)



Apenas S3 (first_review_at). Mann-Whitney: $\text{LOC} \leq 25$ vs $\text{LOC} > 125$. Medianas em minutos. Entre PRs pequenas e grandes, a **mediana** da primeira revisão formal sobe apenas de **~3,6 min** para **~5,7 min**, embora o Mann-Whitney seja significativo ($p < 0,001$). O Spearman entre LOC e latência permanece baixo ($\rho \approx 0,11$). A diferença típica é de poucos minutos — não de horas.

Q2: O TAMANHO DO PR INFLUENCIA A COMPLEXIDADE DA REVISÃO DE CÓDIGO?



O tamanho impacta de forma moderada a duração do processo de integração.

Q2: O TAMANHO DO PR INFLUENCIA A COMPLEXIDADE DA REVISÃO DE CÓDIGO?

Síntese dos resultados

Dimensão	Resultado	Força
M2.3 — volume de comentários	~3,8× mais comentários (pequeno → grande)	Forte
M2.3' — comentários por LOC	~31 × menos densidade (mediana)	Forte (sentido inverso)
M2.4 — 1ª revisão formal	< 6 min na mediana; $p \approx 0,11$	Fraca na prática
M2.5 — até fechar	~3,2× mediana (pequeno → grande)	Moderada

- H_1 vale para volume e ciclo; não vale como “revisão mais profunda por linha”.
- S4 — PR fechada com tempo válido (closed_at — created_at em [0; 8.760] h).
- M2.4 é fraco na mediana — conclusão biforme.

Q3: QUAL A RELAÇÃO DO TAMANHO DO COMMITS NA MANUTENABILIDADE DO CÓDIGO

M3.1: LOC médio por commit

M3.2: Complexidade ciclomatica/LOC

M3.3: Reverts/rollbacks

Hipótese

Commits menores podem estar associados a um ciclo de manutenção mais ágil (menor MTTR), facilitando correções rápidas. Por outro lado, commits grandes podem gerar maior incidência de reverts devido à dificuldade de validação completa da mudança antes do merge.



Q3: QUAL A RELAÇÃO DO TAMANHO DO COMMITS NA MANUTENABILIDADE DO CÓDIGO

- **repo_name e commit_sha:** Atributos de conexão e identidade do artefato.
- **files_changed:** Número de arquivos tocados pelo commit.
- **total_loc_modified:** O volume de linhas tocadas (adicionadas + excluídas).
- **is_revert:** Flag que identifica a natureza de reversão (rollback) a partir das mensagens.
- **is_bug_fix:** Flag extraído via NLP apontando se no commit constava uma correção ou patch de manutenção.
- **cc_max:** Cálculo gerado sobre a árvore da linguagem que mede o pior cenário de caminhos possíveis no código.
- **cc_por_loc:** Densidade da complexidade. Demonstra o quão complexa e condensada é a lógica introduzida e sua escalabilidade em relação ao volume do arquivo.

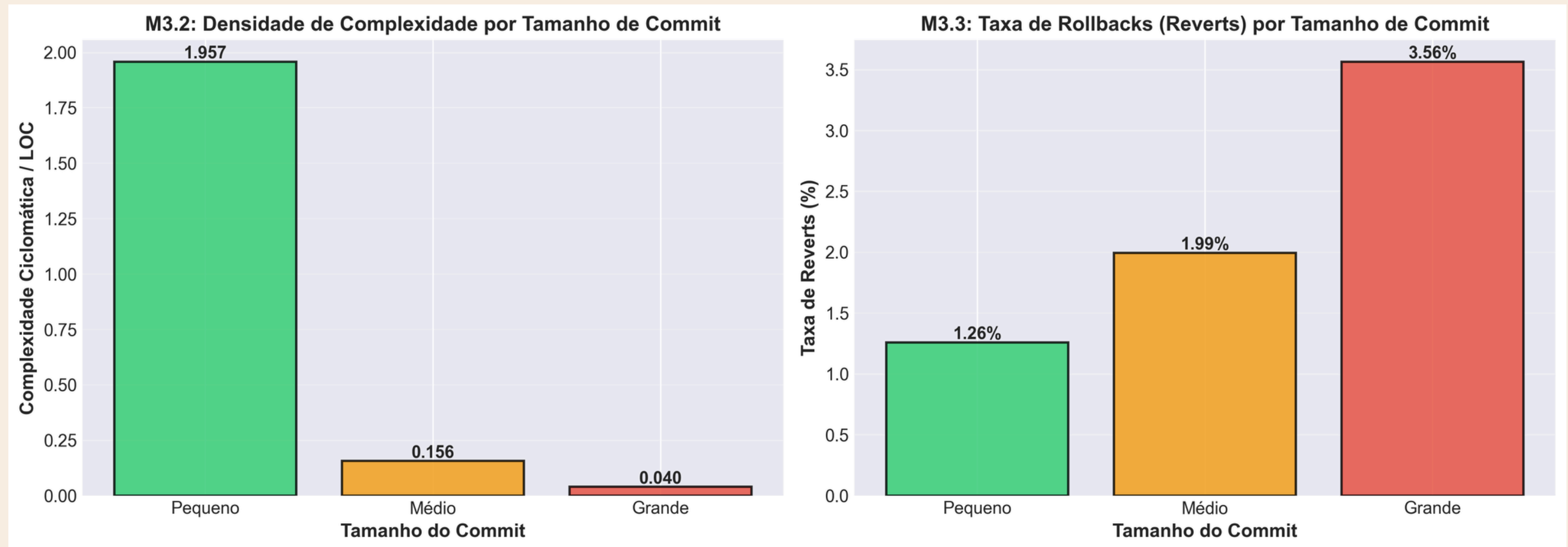


Q3: QUAL A RELAÇÃO DO TAMANHO DO COMMITS NA MANUTENABILIDADE DO CÓDIGO

Tamanho do Commit	Qtd. Commits	Média LOC	Média CC / LOC	Taxa Reverts	Taxa Bug Fix
Pequeno	254.663	7,8	1,96	1,26%	36,18%
Médio	228.527	285,8	0,16	1,99%	35,15%
Grande	77.477	7.512,40	0,04	3,56%	31,15%



Q3: QUAL A RELAÇÃO DO TAMANHO DO COMMITS NA MANUTENABILIDADE DO CÓDIGO



Q3: QUAL A RELAÇÃO DO TAMANHO DO COMMITS NA MANUTENABILIDADE DO CÓDIGO

Hipótese Comprovada: O tamanho do commit tem relação direta e inversamente proporcional à manutenibilidade.

Manutenção Ágil: Commits menores facilitam correções atômicas e rápidas (alto índice de bug fixes, baixo revert).

Alta densidade lógica (CC/LOC alto, média de 1,96), indicando forte concentração e foco (patch direto no problema).

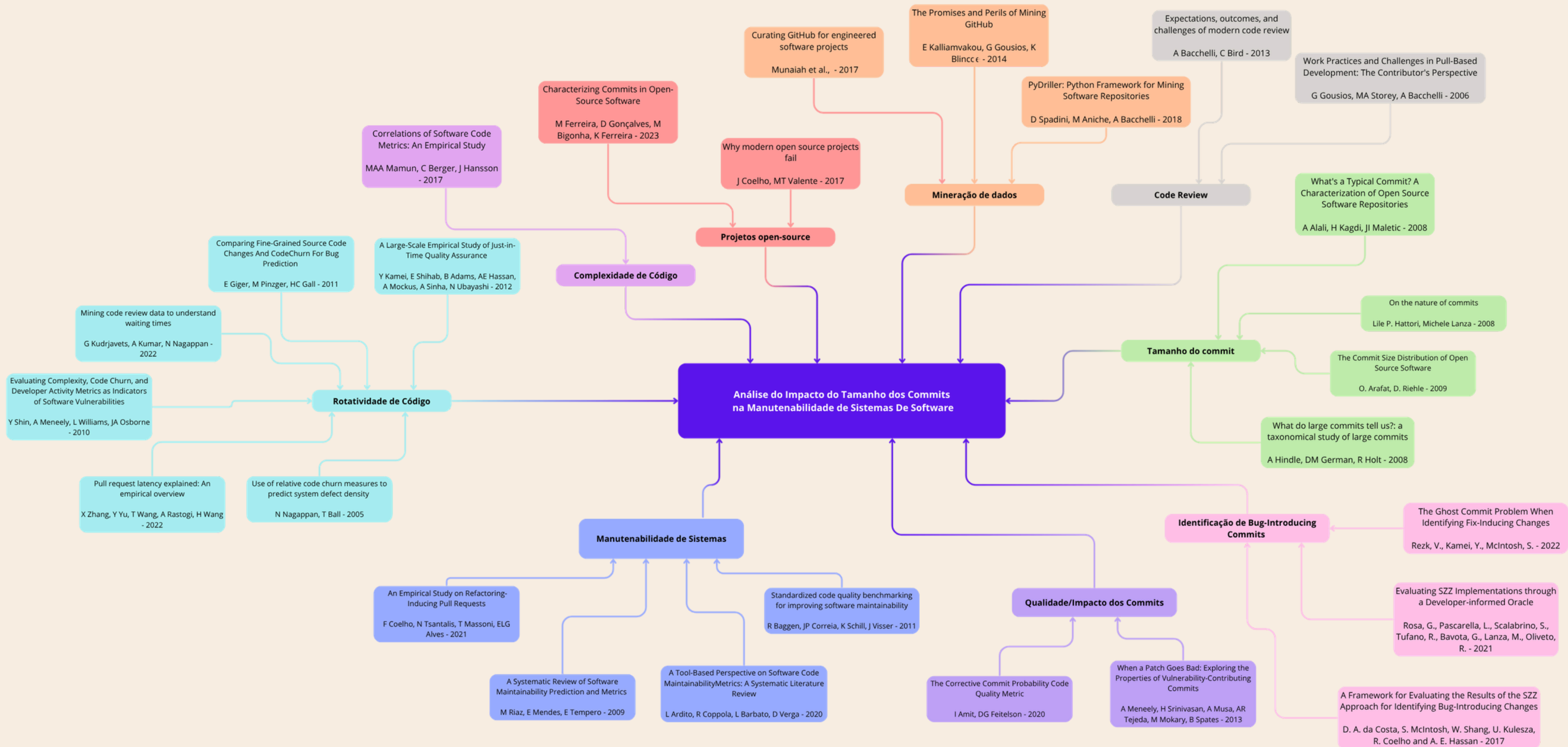
O Risco dos Commits Grandes:

Carga cognitiva extrema no Code Review, mas diluídos em milhares de linhas.

Maior incidência de efeitos colaterais.

Validação incompleta antes do merge, justificando a extrema taxa de código revertido.





REFERÊNCIAS



HATTORI, L. P.; LANZA, M. On the nature of commits. In: 23rd IEEE/ACM International Conference on Automated Software Engineering Workshops, 2008, L'Aquila, Italy. Proceedings... p. 63–71. DOI: 10.1109/ASEW.2008.4686322.

O. Arafat and D. Riehle, "The Commit Size Distribution of Open Source Software," 2009 42nd Hawaii International Conference on System Sciences, Waikoloa, HI, USA, 2009, pp. 1-8, doi: 10.1109/HICSS.2009.421.

Md Abdullah Al Mamun, Christian Berger, and Jörgen Hansson. 2017. Correlations of software code metrics: an empirical study. In Proceedings of the 27th International Workshop on Software Measurement and 12th International Conference on Software Process and Product Measurement (IWSM Mensura '17). Association for Computing Machinery, New York, NY, USA, 255–266. <https://doi.org/10.1145/3143434.3143445>

MUNAIAH, N.; KROH, S.; CABREY, C.; NAGAPPAN, M. Curating GitHub for engineered software projects. Empirical Software Engineering, v. 22, n. 6, p. 3219–3253, 2017.

REFERÊNCIAS



DA COSTA, D. A.; MCINTOSH, S.; SHANG, W.; KULESZA, U.; COELHO, R.; HASSAN, A. E. A Framework for Evaluating the Results of the SZZ Approach for Identifying Bug-Introducing Changes. IEEE Transactions on Software Engineering, v. 43, n. 7, p. 672–692, 2017. DOI: 10.1109/TSE.2016.2616306.

ŚLIWERSKI, J.; ZIMMERMANN, T.; ZELLER, A. When Do Changes Induce Fixes? In: International Workshop on Mining Software Repositories (MSR), 2005, Saint Louis, Missouri, USA. Proceedings... ACM, 2005. DOI: 10.1145/1083142.1083147.

ROMANO, J.; KROMREY, J. D.; CORAGGIO, J.; SKOWRONEK, J.; DEVINE, L. Appropriate statistics for ordinal level data: Should we really be using t-test and Cohen's d for evaluating group differences on the NSSE and other surveys? In: Annual Meeting of the Florida Association of Institutional Research, 2006.

ARCURI, A.; BRIAND, L. A Hitchhiker's guide to statistical tests for assessing randomized algorithms in software engineering. Software Testing, Verification and Reliability, v. 24, n. 3, p. 219–250, 2014. DOI: 10.1002/stvr.1486.



OBRIGADO!

Hora das perguntas, fique à vontade.

